

EX.NO.8

INTER-THREAD COMMUNICATION

PROGRAM:

```
class SharedResource
{
    private int data;
    private boolean available = false;
    public synchronized void produce(int value)
    {
        while (available)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
        data = value;
        System.out.println("Produced: " + data);
        available = true;
        notify();
    }
    public synchronized void consume()
    {
        while (!available)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
        System.out.println("Consumed: " + data);
        available = false;
        notify();
    }
}

class Producer implements Runnable
{
    private SharedResource resource;

    public Producer(SharedResource resource)
    {
        this.resource = resource;
    }

    public void run()
```

```

    {
        for (int i = 1; i <= 5; i++)
        {
            resource.produce(i);
            try
            {
                Thread.sleep(100);
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
    }
}

class Consumer implements Runnable
{
    private SharedResource resource;

    public Consumer(SharedResource resource)
    {
        this.resource = resource;
    }
    public void run()
    {
        for (int i = 1; i <= 5; i++)
        {
            resource.consume();
            try
            {
                Thread.sleep(100);
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public class InterThreadCommunication
{
    public static void main(String[] args)
    {
        SharedResource resource = new SharedResource();
        Thread producerThread =
            new Thread(new Producer(resource));
        Thread consumerThread =
            new Thread(new Consumer(resource));
        producerThread.start();
        consumerThread.start();
    }
}

```

OUTPUT:

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5